Introduction to Computational Complexity

James Hindmarch

January 2024

Good book: Arora & Barak, Computational Complexity: A modern approach

Contents

0 Introduction **3**
0.1 Computational problems . . . 3
0.2 Turing machines . . . 3
0.3 Variants of the Turing machine . . . 4

1 Some complexity classes **4**
1.1 Polynomial Time . . . 4
1.2 Nondeterministic polynomial time . . . 4
1.3 coNP . . . 5
1.4 The polynomial hierachy . . . 5
1.5 PSPACE . . . 6
1.6 EXPTIME . . . 7

2 Circuit Complexity **8**
2.1 Straight-line computation . . . 8
2.2 P/poly . . . 10

3 Search problems and Decision problems **11**

4 Log-Space: L **13**
4.1 Low-depth computation . . . 14

5 Randomized computation **15**
5.1 #P . . . 16

6 Algebraic Complexity **17**

7 Completeness **19**
7.1 Completeness in space classes . . . 22
7.2 A PSPACE-complete problem . . . 23
7.3 NL-completeness . . . 24

7.4 P-completeness 25

8 Less obvious relationships between complexity classes 27

9 Two randomized algorithms 29
9.1 Polynomial identity testing 29
9.2 Primality Testing 30

Lecture 1

0 Introduction

0.1 Computational problems

Example 0.1. Given a graph G with n vertices, and $x, y \in V(G)$, is there a path from x to y?

This problem has a variable input, and an output in $\{0, 1\}$.

Definition 0.2 (Decision Problem)**.** If the output belonging to $\{0, 1\}$ (or if you prefer, $\{\text{no}, \text{yes}\}$), then the problem is called a **decision problem**.

Write $\{0, 1\}^*$ for the set

$$\bigcup_{n=1}^{\infty} \{0, 1\}^n.$$

Then a decision problem can be encoded as a **boolean function**, that is, a function $f : \{0, 1\}^* \to \{0, 1\}$.

Definition 0.3 (Language)**.** The set

$$\{x \in \{0, 1\}^* \, : \, f(x) = 1\}$$

is called the **language** defined by f.

0.2 Turing machines

We will try to avoid talking about turing machines, because implementation details are boring and usually don't matter. Alas, it is impossible to avoid talking about them completely.

Definition 0.4 (Turing machine)**.** A **Turing machine** formalizes the notion of an algorithm. A **k-tape Turing machine** consists of several ingredients:

1. A collection of k tapes where a tape is an infinite sequence of **cells**.
2. A finite set A called the **alphabet**. Each cell contains an element of A.
3. A **head** which is in a **state** (an element of a finite set S of states) and in a **position** in each tape. There are two special states in S: S_{init} and S_{halt}.
4. A **state** is a function that takes as input an element of $A^k \times S$ (the elements in the cells it is looking at, and the state it is in) and outputs $A^k \times S \times \{\text{L}, \text{N}, \text{R}\}$ (it rewrites the current states with the output A^k, then moves into the new state S, and moves the k tapes either one space left, one space right, or not at all).

5. One tape is designated as the **input** tape, and never changed. Another is the output tape. All tapes other than the input tape start full of zeros.
6. If the machine reaches the state S_{halt}, it stops, and if the input is $x \in \{0,1\}^*$ and the output is $y \in \{0,1\}^*$, then we say that the machine computes y, given input x.

0.3 Variants of the Turing machine

We can assume that $A = \{0,1\}$, that $k = 1$ (with a different convention about input-output tapes); that the tapes are 2-sided infinite instead of 1-sided infinite, et cetera.

1 Some complexity classes

We begin with perhaps the most important complexity class.

1.1 Polynomial Time

Definition 1.1 (Polynomial time)**.** The complexity class P consists of all Boolean functions $f : \{0,1\}^* \to \{0,1\}$ such that there exists a Turing machine T, and a polynomial p such that for every x, T computes $f(x)$ in at most $p(|x|)^1$ steps.

Example 1.2. The problem st-CONN (input: directed graph G and two vertices s, t; output 1 if there is a directed path from s to t) belongs to P. The idea would be to find all the vertices that are reachable from s in one step, then those that are reachable in two steps, and so on. Then eventually we will have a list of all the vertices that are reachable from S.

Example 1.3. Input: positive integers m, n, and output mn (remember, the size of the input is not m or n or $m + n$, but more like $\log_2(m + n)$)

Lecture 2

1.2 Nondeterministic polynomial time

Example. A nondeterministic polynomial time problem is to input a graph G with n vertices, and output 1 iff G contains a Hamilton cycle (a cycle that contains every vertex).

Loosely, a nondeterminstic algorit4m is one that doesn't fully specify what it does. It outputs $f(x) = 1$ if and only if there is *some* sequence of choices that leads to $f(x) = 1$.

More formally, we have

Definition 1.4 (Nondeterministic Turing machine)**.** A **nondeterministic Turing machine** is one that has not one, but *two* transition functions. At each step, it applies one or the other.

We say that it computes f if $f(x) = 1$ if and only if there is some sequence of choices that leads to outpt 1 when the input is x.

Definition 1.5 (Nondeterministic polynomial time)**.** NP is the class of functions computable in polynomial time by a nondeterministic Turing machine.

Definition (Nondeterministic polynomial time, Alternative definition)**.** A function $f \in \mathsf{NP}$ if there is a polynomial p, and a function $g \in \mathsf{P}$ such that for every $x \in \{0,1\}^*$, $f(x) = 1$ iff $\exists y \in \{0,1\}^{p(|x|)}$ such that $g(x,y) = 1$.

To see that these are equivalent, observe first that if f satisfies the second definition, then $f \in \mathsf{NP}$, since we can write down y nondeterministically (for $p(|x|)$ steps), and then apply g.

For the other direction, y encodes the choices made by the NDTM (nondeterministic Turing machine), so given y, the computation can be done deterministically.

The big open problem, of course, is whether $\mathsf{P} = \mathsf{NP}$? Probably not.

1.3 coNP

Definition 1.6 (coNP)**.** $f \in \mathsf{coNP}$ iff $\neg f$, or $1 - f$, $\in \mathsf{NP}$.

Alternatively $f \in \mathsf{coNP}$ iff there exists a polynomial p, and $g \in \mathsf{P}$ such that

$$\forall x \in \{0,1\}^* \;\; f(x) = 1 \Leftrightarrow \forall y \in \{0,1\}^{p(|x|)} \;\; g(x,y) = 1$$

1.4 The polynomial hierachy

Definition 1.7 (Σ_n^{P} and Π_n^{P})**.** Define

$$\Sigma_0^{\mathsf{P}} = \mathsf{P} \qquad\qquad \Pi_0^{\mathsf{P}} = \mathsf{P}$$

If Σ_k^{P} and Π_k^{P} have been defined, then $f \in \Sigma_{k+1}^{\mathsf{P}}$ iff there exists a polynomial p and a $g \in \Pi_k^{\mathsf{P}}$ such that

$$f(x) = 1 \text{iff} \exists y. \;\; g(x,y) = 1$$

. Similarly, $f \in Pi_{k+1}^{\mathsf{P}}$ iff there exists a polynomial p and a $g \in \Sigma_k^{\mathsf{P}}$ such that

$$f(x) = 1 \text{iff} \forall y. \;\; g(x,y) = 1.$$

Example 1.8. $f \in \Sigma_5^{\mathsf{P}}$ iff there exists $h \in \mathsf{P}$ such that

$$f(x) = 1 \Leftrightarrow \exists y_1 \forall y_2 \exists y_3 \forall y_4 \exists y_5 \;\; h(x, y_1, y_2, y_3, y_4, y_5).$$

Definition 1.9 (Polynomial Hierarchy)**.** The **polynomial hierarchy** is defined as

$$\mathsf{PH} = \cup_{k=0}^{\infty} \Sigma_k^{\mathsf{P}} \cup \Pi_k^{\mathsf{P}}$$

Proposition 1.10. *If* $\mathsf{P} = \mathsf{NP}$*, then* $\mathsf{P} = \mathsf{PH}$.

Proof. First note that if $\mathsf{P} = \mathsf{NP}$, then $\mathsf{P} = \mathsf{coNP}$. (For a given f, just negate it, calculate it, and negate it back again.)

If $f \in \Sigma_{k+1}^{\mathsf{P}}$, then $\exists g \in \Pi_k^{\mathsf{P}}$ such that $f(x) = 1 \Leftrightarrow \exists y \;\; g(x, y) = 1$. By induction, $g \in \mathsf{P}$, so $f \in \mathsf{NP} \Rightarrow f \in \mathsf{P}$. The proof for Π_{k+1}^{P} is essentially the same, except you wind up in coNP, which is still ultimately P. □

Exercises on Sheet 1: If $\mathsf{NP} = \mathsf{coNP}$ then $\mathsf{PH} = \mathsf{NP} = \mathsf{coNP}$. If $\Sigma_k^{\mathsf{P}} = \Sigma_{k+1}^{\mathsf{P}}$, or $\Sigma_k^{\mathsf{P}} = \Pi_k^{\mathsf{P}}$, then $\mathsf{PH} = \Sigma_k^{\mathsf{P}}$. So if there is ever equality, then the polynomial hierarchy collapses down to that point. This is taken as some weak evidence that there isn't ever equality, because people don't believe the polynomial hierarchy should collapse.

1.5 PSPACE

Definition 1.11 (PSPACE)**.** PSPACE consists of functions that can be computed by a Turing machine that uses only a polynomial amount of tape.

Proposition 1.12.

$$\mathsf{NP} \subseteq \mathsf{PSPACE}.$$

Proof. If $f(x) = 1 \Leftrightarrow \exists y. \;\; g(x, y) = 1$, and $g \in \mathsf{P}$, then to compute f by a brute-force search, all you need to remember is which y you have gotten to. You can do this by just incrementing a binary y by 1, and also as soon as you have found a y such that $g(x, y) = 1$. You can *reuse* the space that you needed to compute each $g(x, y)$, and $g(x, y)$ never needs more than polynomial space, since it can be computed in polynomial time. □

Exercise on sheet: $\mathsf{PH} \subset \mathsf{PSPACE}$. The typical way to find something in PSPACE which is not in PH is to look at games, since for winning strategies in games, we need to alternate quantifiers a lot (i.e. there exists a move that I can play such that for all moves that you then play there will exist a move that I can play such that for all moves that you then play there exists a ...).

1.6 EXPTIME

Definition 1.13 (EXPTIME)**.** EXPTIME is the class of functions that can be computed in time $\exp\big(O(n^k)\big)$ for some k.

Lecture 3

Proposition 1.14.

$$\mathsf{PSPACE} \subset \mathsf{EXPTIME}$$

Proof. Given a Turing machine in the middle of a computation, define its **configuration** to be its state, its position on each tape, and the values in all the cells on those tapes.

If T uses only a polynomial amount of space $p(n)$ for an input of size n, and has k tapes, then the number of possible configurations is at most

$$|S| \times p(n)^k \times |A|^{k \times p(n)},$$

where A is the alphabet. So if the computation goes on for longer than this, then the configuration must repeat by the pigeonhole principle, and so is eventually periodic and will never halt. □

We also have the complexity classes:

Definition 1.15 (NEXPTIME)**.** For a function f, we have:

$$f \in \mathsf{NEXPTIME} \Leftrightarrow \exists g \in \mathsf{EXPTIME} \;\; f(x) = 1 \text{ iff } \exists y.\, g(x, y) = 1$$

and

Definition 1.16 (NEXPSPACE)**.** A function f is in NEXPSPACE if and only if f can be computed using at most $\exp(p(n))$ space for an input of size n, and some polynomial $p(n)$.

We have

$$\mathsf{P} \subset \mathsf{NP} \subset \mathsf{PSPACE} \subset \mathsf{EXPTIME} \subset \mathsf{NEXPTIME} \subset \mathsf{NEXPSPACE}$$

We do not know if any of these are proper, but we do know that

$$\begin{aligned} \mathsf{P} &\subsetneq \mathsf{EXPTIME} \\ \mathsf{NP} &\subsetneq \mathsf{NEXPTIME} \\ \mathsf{PSPACE} &\subsetneq \mathsf{NEXPSPACE} \end{aligned}$$

So if you jump 3 then we know the inclusions are proper, but not for any amount less than that.

2 Circuit Complexity

Definition 2.1 (Circuit)**.** A **Circuit** is a directed acyclic graph (DAG) such that each vertex is labelled as either

- An input,
- An output,
- An AND gate,
- An OR gate,
- or a NOT gate.

An input is a vertex of in degree 0. A NOT gate is a vertex of in degree 1. ALl vertices of in degree > 1 are AND gates or OR gates. If the vertex has out-degree 0, then it is an output.

- If the vertex preceding a NOT gate has value x, then the vertex at the NOT gate has value $1 - x$.
- The value at an AND (OR) gate is the minimum (maximum) of the values of its predecessors.

Using these rules, we get a well-defined function from $\{0,1\}^I$ to $\{0,1\}^O$ where I is the set of inputs, and O is the set of outputs.

If every AND gate and OR gate has in-degree $\leqslant k$, then we say that the circuit is of **fanin** $\leqslant k$. Often we restrict to circuits of fanin $\leqslant 2$.

2.1 Straight-line computation

Definition 2.2 (Straight-line computation)**.** Let $f : \{0,1\}^n \to \{0,1\}$. A **straight-line computation** of f is a sequence of functions $f_1, \ldots, f_m$ (where

m is the **length** of the computation) such that $f_i(x) = x_i$ for $1 \leqslant i \leqslant n$, and for each $i > n$ either

$$f_i = f_{j_1} \wedge \cdots \wedge f_{j_k}$$

for some $j_1, \ldots, j_k < i$; or

$$f_i = f_{j_1} \vee \cdots \vee f_{j_k}$$

for some $j_1, \ldots, j_k < i$; or $f_i = 1 - f_j$ for some $j < i$, and $f_m = f$.

By considering a total ordering on the vertices of a DAG such that if $v_1 \to v_2$, then $v_1 < v_2$, we see that the smallest circuit that computes f is the smallest length of a straight-line computation.

Lemma 2.3. *Every function*

$$f : \{0,1\}^n \to \{0,1\}$$

can be computed by a circuit of size at most exponential in n.

Proof. Left as an exercise.

Proposition 2.4. *Let f be a function that can be computed by a Turing machine T in time $t(n)$ for inputs of size n. Then there is a family (C_n) of circuits such that $|C_n| = O(t(n)^{k+2})$ where k is the number of tapes of T, and C_n computes f for inputs of size n.*

Proof. Let $S = \{s_1, \ldots, s_r\}$ be the set of states of T. (Assume that the alphabet is $\{0,1\}$). Then we can encode a configuration of T at time t using variables $\sigma_1(t), \ldots, \sigma_r(t)$, and $\pi_i^h(t)$ for $1 \leqslant i \leqslant t(n)$, $1 \leqslant h \leqslant k$, and $v_i^h(t)$.

We have:

- $\sigma_i(t) = 1$ iff T is in state s_i at time t.
- $\pi_i^h(t) = 1$ iff the head is at position i on tape h at time t.
- $v_i^h(t) =$ value in cell i of tape h at time t.

Note that $\sigma_i(t) = 1$ iff there exist $j \leqslant r$ and $i_1, \ldots, i_k$ such that $\sigma_j(t-1) = 1$, and $\pi_{i_h}^h(t-1) = 1$ for $h = 1, \ldots, k$, and the transition function of T, $\tau(s_j, v_{i_1}^1, \ldots, v_{i_k}^k)$, has state component equal to s_i.

So for any given $i_1, \ldots, i_k$, we have a function of $k+1$ variables to evaluate, which we can do with a circuit of bounded size in terms of h. Hence, we can

Lecture 4 calculate $\sigma_i(t)$ from the previous configuration in time $O(t(n)^k)$. Similarly, we can compute each position variable $\pi_i^h(t)$ from the configuration at time $t-1$ with a circuit of size $O(t(n)^k)$, and also each state variable $v_j(t)$. So we can compute the configuration at time t from the configuration at time $t-1$ with a circuit of size $O(t(n)^{k+1})$, since there are at most $t(n)$ of these variables, so the result follows because there are at most $t(n)$ configurations. □

2.2 P/poly

This complexity class has three definitions:

Definition 2.5 (P/poly)**.** $f \in \mathsf{P/poly}$ if one of the following (and hence all) holds:

1. There is a family (C_n) of polynomial-sized circuits such that C_n computes $f(x)$ when $|x| = n$. So given proposition 2.4, it is easy to see that $f \in \mathsf{P} \Rightarrow f \in \mathsf{P/poly}$.
2. There is a polynomial p, and a sequence (y_n) with $|y_n| = p(n)$ and a function $g \in \mathsf{P}$ such that
$$f(x) = 1 \Leftrightarrow g(x, y_{|x|}) = 1.$$
3. There is a sequence (T_n) of Turing machines and a polynomial p such that T_n has at most $p(n)$ states, and T_n computes $f(x)$ when $|x| = n$.

Definition 2.6 (P-uniform)**.** A sequence of (C_n) of circuits is **P-uniform** if there is a polynomial-time algorithm that generates it. (I.e. given input 1^n, it outputs C_n).

Proposition 2.7. *These definitions of* P/poly *are all equivalent.*

Proof.

(1) ⇒ (2) Let y_n be an encoding of C_n and let $g(x, y) = 1$ if the circuit encoded by y outputs 1 with input x. Here, g is the circuit evaluation function, which is in poly by prop 2.4.

(2) ⇒ (1) Fix an input size n, and have a C (depending on n) computing g_n, and let C_n be C with the last $p(n)$ inputs restricted to y_n.

(2) ⇒ (3) Fix n and let T compute g_n, and let T_n be a turing machine that prints out y_n, and then uses T to compute $g(x, y_n)$.

(3) $\Rightarrow$ (2) Let y_n be an encoding of T_n, and let $g(x, y_n) = 1$ if the turing machine encoded by y_n outputs 1 with input x.

□

3 Search problems and Decision problems

Let g be a boolean function of two variables. Then we get the decision problem "does there exist y such that $g(x, y) = 1$?" and a corresponding **search problem**: "if there exists y such that $g(x, y) = 1$, then find one." A solution to the search problem is an algorithm that outputs y such that $g(x, y) = 1$ if it exists.

Proposition 3.1. *If $\mathsf{P} = \mathsf{NP}$, and $f \in \mathsf{NP}$ so that $f(x) = 1$ iff there exists y such that $g(x, y) = 1$ for $g \in \mathsf{P}$, then there is a polynomial-time algorithm that comnputes $h : \{0,1\}^* \to \{0,1\}^*$ such that if $f(x) = 1$ then $g(x, h(x)) = 1$.*

Proof. You just play a game of 20 questions. If $\mathsf{P} = \mathsf{NP}$, we can solve decision problems as much as we want, so ask if there exists such a y, then ask whether it is possible for the first digit of y to start with 1, then the second digit, then the third digit etc.

So for each i, let g_i be the function that takes as input x and u_i where $|u_i| = i$, and outputs 1 iff $\exists v$, $|v| = p(|x|) - i$ such that $g(x, u, v) = 1$. (where u, v is a splitting up of y). Now run the following procedure: Calculate $g_1(x, 1)$ in polynomial time, since $\mathsf{P} = \mathsf{NP}$. Let $u_1 = g_1(x, 1)$. Note that if there exists a y such that $g(x, y) = 1$, then there exists a v such that $g_1(x, u_1, v) = 1$. Now let $u_2 = U_1$, $g_2(x, u_1)$, $u_3 = u_2$, $g_3(x, u_2)$, and so on. □

Lemma 3.2. *If $\mathsf{NP} \subset \mathsf{P/poly}$, then for $f \in \mathsf{NP}$, there is a polynomial-sized family of circuits (C_n) such that if $|x| = n$, then C_n with input x computes y such that $g(x, y) = 1$.*

Proof. For each i, since $\mathsf{NP} \subset \mathsf{P/poly}$, there is a polynomial-sized circuit C'_i that computes g_i. Now put together the circuits C'_1 up to $C'_{p(n)}$ as follows:

- C'_1 takes inputs $x_1, \ldots, x_n, 1$ and outputs u_1.
- C'_2 takes inputs $x_1, \ldots, x_n, u_1, 1$ and outputs u_2. ⋮
- Until we reach C'_n, then the output if there exists y such that $g(x, y) = 1$ will be such a y.

□

Theorem 3.3 (The Karp-Lipton Theorem)**.** *If* $\mathsf{NP} \subset \mathsf{P/poly}$ *then* $\Sigma_2^{\mathsf{P}} = \Pi_2^{\mathsf{P}}$ *(and therefore* $\mathsf{PH} = \Sigma_2^{\mathsf{P}} = \Pi_2^{\mathsf{P}}$*).*

Lecture 5

Proof. Let $f \in \Pi_2^{\mathsf{P}}$, and let $h \in \mathsf{P}$ be such that

$$f(x) = 1 \text{ iff } \forall y \, \exists z. \, h(x, y, z) = 1,$$

where y, z are of "appropriate" polynomial size, depending on $|x|$. Now define $g(x, y)$ to be 1 iff $\exists z h(x, y, z) = 1$. Now this function g belongs to NP, so by our hypothesis it is in $\mathsf{P/poly}$, and so by lemma 3.1 there must be a circuit family (C_n) of polynomial size such that for every x, if $|x| = n$ and $g(x, y) = 1$, then

$$h(x, y, C_n(x, y)) = 1$$

where $C_n(x, y) =$ output of C_n with input x, y. So

$$f(x) = 1 \Rightarrow \exists C_n \, \forall y. \, h(x, y, C_n(x, y)) = 1.$$

Checking whether $h(x, y, C_n(x, y)) = 1$ can be done in polynomial time. However, we need this to be an if and only if for f to be in Σ_2^{P}. If $f(x) = 0$, then

$$\exists y \, \forall z. \, h(x, y, z) = 0,$$

so the $\Rightarrow$ is in fact a $\Leftrightarrow$. Therefore $f \in \Sigma_2^{\mathsf{P}}$. By doing the same for $1 - f$, we get the reverse implication. □

Lemma 3.4. *For every* k*, there is a boolean function* f *that can be computed by a circuit family of size* n^{k+1} *but not by a circuit family of size* n^k*.*

Proof. On example sheet 1. □

Theorem 3.5. *For every* k*, there is a boolean function* $f \in \Sigma_4^{\mathsf{P}}$ *that cannot be computed by a family of circuits of size* n^k*.*

Proof. For n sufficiently large, the lemma gives us

$$f'_n : \{0, 1\}^n \to \{0, 1\}$$

that can be computed by a circuit of size n^{k+1} but not by a circuit of size n^k. Choose a sensible ordering $(\prec)$ on circuits of size $\leqslant n^{k+1}$. If $|x| = n$, then let $f_n(x)$ be $C_n(x)$ where Cn is the first circuit (in this ordering) such that

$|C_n| \leqslant n^{k+1}$ and no circuit of size $\leqslant n^k$ computes the same function as C_n (which is guaranteed to exist by lemma 3.4). Then let

$$f = (f_n)_1^\infty.$$

If $n < n_0$, f_n arbitrary.

Then if $|x| = n$, we have that $f(x) = 1$ iff there exists a circuit C_n such that $|C_n| \leqslant n^{k+1}$ and for all circuits D where $|D| \leqslant n^k$, there exists y such that $C_n(y) \neq D(y)$ *and* also for all E with $E \prec C_n$, there is some circuit F with $|F| \leqslant n^k$ so that for all z, $E(z) = F(z)$ *and also* that $C_n(x) = 1$. This is basically how we constructed C_n. So we have a $\exists\forall\exists\forall(\forall)$ statement, so $f \in \Sigma_4^\mathsf{P}$. □

Corollary 3.6 (Kannan)**.** *In fact, for every k, there is a function $f \in \Sigma_2^\mathsf{P} \cap \Pi_2^\mathsf{P}$ that cannot be computed by a circuit family of size n^k.*

Proof. By the Karp-Lipton theorem, if $\mathsf{NP} \subset \mathsf{P/poly}$, then $\mathsf{PH} \subset \Sigma_2^\mathsf{P} \cap \Pi_2^\mathsf{P}$, so the function just defined does the job.

However, if $\mathsf{NP} \not\subset \mathsf{P/poly}$, then we get the stronger result that there is $f \in \Sigma_1^\mathsf{P} (= \mathsf{NP})$ (and therefore in $\Sigma_2^\mathsf{P} \cap \Pi_2^\mathsf{P}$) that can't be computed by any circuit family of polynomial size. □

4 Log-Space: L

Definition 4.1 (L)**.** Roughly L is the set of functions that can be computed with a logarithmic amount of memory.

More formally $f \in \mathsf{L}$ if there is a turing machine that computes f with a read-only input tape that contains the input and cannot be modified, and a work-tape of size $O(\log n)$ for input of size n.

(If you want several outputs instead of just one, then we must also restrict to a write-only output tape.)

Definition 4.2 (NL)**.** It's like L, except the turing machine is nondeterministic.

We can also give a certificate definition of NL, but we have to be careful. We say that $f \in \mathsf{NL}$ iff there is a Turing machine with a read-only input tape, a work tape of size $O(\log n)$, and a read-once certificate tape on which the head can only ever stay still or move right, such that $f(x) = 1$ iff there

exists some y (of length polynomial in $|x|$) that can be put in the certificate tape such that the Turing machine outputs 1.

Definition 4.3 (Configuration Graph)**.** The **configuration graph** of a Turing machine is a directed graph where the vertices are the configurations, and C is joined to C' if it is possible for T to go from configuration C to configuration C' in one step. For a non-deterministic turing machine, there will be two "out" edges for each vertex.

Proposition 4.4.

$$\mathsf{NL} \subset \mathsf{P}$$

Lecture 6

Proof. Let T be a nondeterministic Turing machine that computes a function $f \in \mathsf{NL}$ and uses a logarithmic amount of space. Let G be the **configuration graph** of T. Then $f(x) = 1$ iff there is a directed path in G from the initial configuration to one such that T has halted with output 1. Since T uses $O(\log n)$ space, the number of vertices of G is polynomial in n (since the number of vertices is $\leqslant 2^{O(\log n)} \cdot |S|$ where S is the number of states, which ends up being polynomial in n). But reachability, the problem of determining whether there is a directed path from a vertex to a subset A of vertices in a directed graph, is easily seen to be in P. □

4.1 Low-depth computation

Definition 4.5 (NC^i)**.** The class NC^i ($i \in \mathbb{N} \cup \{0\}$) cnsists of all functions that can be computed by circuits of polynomial size, fanin 2, and depth $O((\log n)^i)$, when the depth of a circuit is the length of the longest directed path in the associated DAG.

$$\mathsf{NC} = \cup_{i=0}^{\infty} \mathsf{NC}^i.$$

Definition 4.6 (AC^i)**.** The class AC^i is very similar to NC^i, but the fanin of the circuits is allowed to be unbounded.

$$\mathsf{AC} = \cup_{i=0}^{\infty} \mathsf{AC}^i.$$

It is an exercise on the sheet to show that $\mathsf{AC}^i \subset \mathsf{NC}^{i+1}$. This obviously makes $\mathsf{NC} = \mathsf{AC}$.

It is usual to impose a uniformity condition on NC^i. The favoured condition is log-space uniformity. I.e. $f \in u - \mathsf{NC}^i$ if the circuits can be generated in log-space.

Whether $\mathsf{P} \subset \mathsf{NC}$ or not is a major open problem. It seems like "probably not." But it is not even known whether $\mathsf{PH} \subset \mathsf{NC}^1$.

5 Randomized computation

Definition 5.1 (RP)**.** A function $f \in \mathsf{RP}$ (randomized polynomial time) if there is a polynomial p, and a function $g \in \mathsf{P}$ such that if $|x| = n$, and $m = p(n)$, then

$$\mathbb{P}_{y \in \{0,1\}^n}\left[g(x,y) = 1\right] = \begin{cases} 0, & f(x) = 0 \\ \geqslant 1/2, & f(x) = 1. \end{cases}$$

So we get no false positives, but we may get false negatives. Note that if we run the computation on independent strings $y_1, \ldots, y_k$, then

$$\mathbb{P}\left[\exists i.\, g(x,y_i) = 1\right] = \begin{cases} 0, & f(x) = 0 \\ 1 - 2^{-k}, & f(x) = 1. \end{cases}$$

So we can make the error probability very small without much cost.

Naturally, we also write $f \in \mathsf{coRP}$ if $1 - f \in \mathsf{RP}$, and ZPP (zero-error probabilistic polynomial time) $= \mathsf{RP} \cap \mathsf{coRP}$.

Definition 5.2 (BPP)**.** We say that a function is in BPP if there exists g as above such that

$$\mathbb{P}_{y \in \{0,1\}^m}\left[g(x,y) = f(x)\right] \geqslant 2/3.$$

BPP stands for bounded-error probabilistic polynomial time.

Like in RP, we can shrink the error probability for $f \in \mathsf{BPP}$, to do so, pick k and calculate $g(x,y_1), \ldots, g(x,y_k)$, (where $y_1, \ldots, y_k$ are independent random elements of $\{0,1\}^m$) and take the majority. The probability of getting the wrong answer is at most $e^{-k/48}$ by Chernoff estimates for the sum of independent Bernoulli random variables.

Corollary 5.3.

$$\mathsf{BPP} \subset \mathsf{P/poly}$$

Proof. If $k \geqslant 48n$, then the probability that the majority of $g(x, y_i)$ is wrong is 2^{-n}. Therefore, there exist $y_1, \ldots, y_k$ such that for every x, the majority vote is correct (since for them all to be incorrect, we're taking a union of bad events, we have 2^{-n} possible ways for... rewatch this section later). This $y_1, \ldots, y_k$ serves as an advice string that works for any x of the same length, together with the function that computes the majority vote. □

Theorem 5.4 (Sipser-Lauterman)**.**

$$\mathsf{BPP} \subset \Sigma_2^{\mathsf{P}} \cap \Pi_2^{\mathsf{P}}.$$

Lecture 7

Proof. If $f \in \mathsf{BPP}$, then there exists a polynomial p and a function $g \in \mathsf{P}$ such that if $|x| = n$, then

$$\mathbb{P}_{y \in \{0,1\}^n} \left[g(x, y) = f(x) \right] \geqslant 1 - 2^{-n}$$

(by the boosting argument given earlier). Thenf or each x, let $A_x \subset \{0,1\}^{p(n)}$ be $\{y : g(x, y) = 1\}$. Then if $f(x) = 1$, A_x has density $\geqslant 1 - 2^{-n}$ and if $f(x) = 0$, then A_x has density $\leqslant 2^{-n}$. Suppose that $f(x) = 1$, and let $y_1, \ldots, y_r$ be chosen independently and uniformly from $\{0,1\}^{p(n)}$. For each y,

$$\mathbb{P}\left[y \notin \bigcup_{i=1}^{r} A_x \oplus y_i \leqslant 2^{-rn} \right]$$

(where $\oplus$ adds vectors pointwise, modulo 2.) because for each y_i the probability is $\leqslant 2^{-n}$ and we have r of them chosen independently. So if we choose $r > \frac{p(n)}{n}$, then this probability is less than $2^{-p(n)}$, so we get that $f(x) = 1 \Rightarrow \exists y_1, \ldots, y_r$ such that $\bigcup_{i=1}^{r} A_x \oplus y_i = \{0,1\}^{p(n)}$.

Equivalently, $f(x) = 1$ means that $\exists y_1, \ldots, y_r$ such that $\forall y \in \{0,1\}^{p(n)}$, $\exists i$ with $g(x, y \oplus y_i) = 1$[2]. But if $f(x) = 0$, then A_x has density of $\leqslant 2^{-n}$, and r is polynomial in n, so for sufficiently large n, it is not possible that $\bigcup_{i=1}^{r} A_x \oplus y_i = \{0,1\}^{p(n)}$. So the implication is $\Leftrightarrow$. Therefore $f \in \Sigma_2^{\mathsf{P}}$. By the symmetry of BPP, $f \in \Pi_2^{\mathsf{P}}$. □

A major open problem is: Does $\mathsf{BPP} = \mathsf{P}$?

5.1 #P

This is a computation class for **counting problems**. For example, instead of determining whether a graph has a Hamilton cycle, it is the problem of determining how many Hamilton cycles a graph has.

[2] and $\exists i$ with $g(x, y \oplus y_i) = 1$ is in P.

Definition 5.5 (#P)**.** A function $f : 0,1^n \to \mathbb{N} \cup \{0\}$ belongs to #P if there exists a polynomial p, and $g \in \mathsf{P}$ such that for every $x \in \{0,1\}^*$:

$$f(x) = |\{y \in \{0,1\}^{p(|x|)} \ : \ g(x,y) = 1\}$$

Note that if we can count solutions, then we can detect whether solutions exist, so #P is at least as hard as NP.

Examples.

- Input a graph G. Output the number of hamilton cycles of G.
- Input a bipartite graph with vertex sets of the same size, and output the number of perfect matchings. This is a really important example. If G is such a graph with two copies of $[n] = \{0, 1, \ldots, n\}$ as its vertex sets, and

  $$A_{ij} = \begin{cases} 1 & \text{if } ij \in E(G) \\ 0 & \text{o.w.} \end{cases}$$

  Then the number of perfect matchings is

  $$\sum_{\sigma \in S_n} \Pi_{i=1}^n A_{i\sigma(i)}$$

  This formula looks like the determinant, but there's no term for signs. We call this the **permanent** of the $n \times n$ matrix A. This is very interesting, because determining the permanent in polynomial time is #P-complete (even for $\{0,1\}$ matrices). However, determining whether there *exists* a matching in a bipartite graph is a polynomial-time problem. So this is a problem where the existence is in P, but counting is #P-complete.

6 Algebraic Complexity

Algebraic complexity is about building up multivariable polynomials using $+$ and $\times$.

Definition 6.1 (Arithmetic circuit)**.** An **arithmetic circuit** is a directed acyclic graph where each vertex is either an input, a $+$-gate or a $\times$-gate (we get subtraction by multiplying by -1). The inputs are constraints from some field $\mathbb{F}$, and variables $x_1, \ldots, x_n$. As with Boolean circuits, we can talk instead about straight-line computations.

The **degree** of a circuit is defined inductively:

- The degree of an input vertex is 0 if it is a constant, and 1 if it is a variable.

- The degree at a $+$-gate is the maximum of the degrees of its inputs.
- The degree of a $\times$-gate is the sum of the degrees of its inputs.

Definition 6.2 (VP)**.** VP is the class of families (p_n) of polynomials, such that p_n can be computed by an arithmetic circuit of polynomial size and polynomial degree. (We need polynomial degree because otherwise you can get x^{2^n} in a linear-size circuit by repeated squaring, which is a ludicrously high degree polynomial).

Example. The determinant of a matrix $(x_{ij})_{i,j=1}^n$ is in VP. (Proof later, it's nontrivial because we aren't allowed to divide).

Definition 6.3 (VNP)**.** A polynomial family (f_n) is in VNP if there is a polynomial p and a polynomial family $(g_n) \in \mathsf{VP}$ such that

$$f_n(x) = \sum_{y=(y_1,\ldots,y_m)\in\{0,1\}^m} g_n(x,y),$$

where $m = p(\#\text{ variables in } x)$.

Example. The permanent belong to VNP.

Proof. We need to find a suitable way of representing the polynomial

$$\sum_{\pi\in S_n} \Pi_{i=1}^n x_{i\pi(i)}.$$

What we're going to do is take products

$$\prod_{i=1}^n \left(\sum_{j=1}^n x_{ij} \right).$$

But the problem with this is we will have many products of x_{ij} which are not of the form that we ultimately want, so instead we will take:

$$\sum_{y\in\{0,1\}^{n^2}} \prod_{i=1}^n \left(\sum_{j=1}^n x_{ij}y_{ij} \right).$$

This still doesn't quite solve our problem, so for each i, we let

$$r_i(y) = \sum_{j=1}^n y_{ij} \prod_{j\neq j'} (y_{ij} - y_{ij'}).$$

Note that $r_i(y) = 1$ iff there is exactly one j such that $y_{ij} = 1$. Now let $r(y) = \prod_{i=1}^{n} r_i(y)$. Similarly, define a polynomial $c(y)$ that takes the value 1 iff y has exactly one 1 in each column. Then

$$\sum_{y \in \{0,1\}^{n^2}} \prod_{i=1}^{n} \left(\sum_{j=1}^{n} x_{ij} y_{ij} \right) r(y) c(y)$$

givs the permanent of x, and gives a formula which has a polynomial size, and a polynomial degree, arithmetic circuit. □

7 Completeness

For many complexity classes, there are problems in the classes that can be shown to be "at least as hard" as any other problem in the class. First of all we will see the most important (and famous) completeness result.

Definition 7.1 (Polynomial-time redcution)**.** Let f and g be Boolean functions. A function $h : \{0,1\}^* \to \{0,1\}^*$ is a **polynomial-time reduction** from g to f if h can be computed in polynomial time and $g = f \circ h$. Therefore, if $f \in \mathsf{P}$, and g can be polynomial-time reduced to f, then $g \in \mathsf{P}$. We sometimes write $g \prec_{\mathsf{P}} f$. Note that the relation $\prec_{\mathsf{P}}$ is transitive.

Definition 7.2 (NP-hard/NP-complete)**.** A function f is said to be **NP-hard** if $g \prec_{\mathsf{P}} f$ for every $g \in \mathsf{NP}$. f is **NP-complete** if f is NP-hard and $f \in \mathsf{NP}$.

Example. (Boring Example). The input is a Turing machine T, some $x \in \{0,1\}^*$ and two unary sequences 1^m and 1^t. Then $\phi(T, x, 1^m, 1^t) = 1$ iff there exists y with $|y| = m$ such that $T(x, y)$ halts on input (x, y) in time $\leqslant t$ and outputs 1. Clearly ϕ is in NP. If $f \in \mathsf{NP}$ and p, q are polynomials, and g is a function in P such that $f(x) = 1$ iff there exists y in $\{0,1\}^{p(|x|)}$ such that $g(x, y) = 1$, then let T, q be such that T computes $g(x, y)$ in time $q(|x|)$. Then $f(x) = 1$ iff $\phi(T, x, 1^{p(|x|)}, 1^{q(|x|)})$. So every $f \in \mathsf{NP}$ can be reduced to this problem easily. We can call this problem TSAT for Turing machine satisfiability (this isn't a standard name).

Example. (CIRCUITSAT). The input here is a circuit C with $n + m$ inputs, and some $x \in \{0,1\}^n$. The output is 1 if and only if there exists $y \in \{0,1\}^m$ such that $C(x, y) = 1$. We can reduce TSAT to CIRCUITSAT as follows:

Given an instance $(T, x, 1^m, 1^t)$ of TSAT, build a circuit C_T that emulates T for t steps with inputs of size $|x| + m$. (If you look at the proof that $\mathsf{P} \subset \mathsf{P/poly}$, you will see not just that such a circuit exists, but also that it can be built by

an algorithm in polynomial time). Then $\phi(T, x, 1^m, 1^t) = 1$ if and only if there exists y such that $C_t(x, y) = 1$. This completes the reduction.

For the next example, we need some definitions. A **variable** is a symbol, x, that stands for an element of $\{0, 1\}$ (or if you want, $\{\text{FALSE}, \text{TRUE}\}$). A **literal** is either x or $\neg x$ for some variable x. A **clause** is an expression of the form $u_1 \vee \cdots \vee u_k$ where $u_1, \ldots, u_k$ are literals. A **formula** (in CNF) is a conjunction of clauses, e.g. $x_1 \wedge (x_2 \vee \neg x_5) \wedge (x_1 \vee x_5)$. A formula is **satisfiable** if it is possible to choose values for the variables such that ϕ evaluates to 1.

Example. (SAT). The SAT problem takes a formula as input, and outputs 1 iff it is satisfiable. 3SAT is the special case where each clause has size at most 3. We shall reduce CIRCUITSAT to 3SAT. First, let $v_1, \ldots, v_m$ be some straight-line computation (assume fanin 2). For each v_i, we take a variable, which we shall also call v_i.

- If $v_i = \neg v_j$ for some $j < i$, then put in clauses $(\neg v_i \vee \neg v_j)$ and $(v_i \vee v_j)$.
- If $v_i = v_j \wedge v_k$ for some $j, k < i$, then put $\neg v_i \vee v_j$, $\neg v_i \vee v_k$, and $\neg v_j \vee \neg v_k \vee v_i$.
- If $v_i = v_j \vee v_k$ for some $j, k < i$, then put in $\neg v_j \vee v_i$, $\neg v_k \vee v_i$, $\neg v_i \vee v_j \vee v_k$.

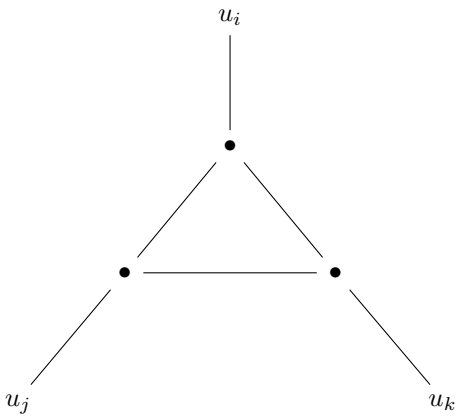
So far, if $v_1, \ldots, v_m$ satisfy the formula, then they must take values corresponding to the straight-line computation. Now add the clause v_m for force the output to be 1. Finally, given C, x, set the inputs corresponding to x to be those values and simplify the formula to some ϕ_x which is satisfiable iff $\exists y$ such that $C(x, y) = 1$.

Lecture 8

Example. (VERTEX COVER). The input here is a graph G and a positive integer m, and the output is 1 if and only if there is a subset $A \subset V(G)$, $|A| = m$, so that every edge has a vertex in A. (So it is a bunch of vertices that touch every edge). To see that this is NP-complete, we shall reduce 3SAT to it. We do this by using a particular "gadget." Let φ be a formula in $x_1, \ldots, x_n$ of 3SAT (For convenience suppose WLOG that all clauses have size exactly 3). For each x_i, build a "variable gadget:"

$$x_i \text{ ——— } \neg x_i$$

which is an edge with vertices labelled x_i and $\neg x_i$. For each clause $u_i \vee u_j \vee u_k$, build a "clause gadget" as follows:



i.e. take a triangle, and connect the three vertices to the vertices labelled u_i, u_j, u_k. Observe that every vertex cover of the resulting graph has size at least $n + 2m$ (n for choosing each x_i and each $\neg x_{i+1}$, and $2m$ for choosing 2 edges from each triangle in the m clauses). Then we claim that there is a vertex cover of size $n + 2m$ iff φ is satisfiable.

To prove this, suppose φ were satisfiable, then let $u_1, \ldots, u_n$ be a satisfying assignment, and pick u_i from i^{th} variable gadget, and then for any clause gadget, we have at least one of the outer edges covered, so we can cover the rest with 2 vertices. If φ is not satisfiable, then for any choice of $u_1, \ldots, u_n$, there will be some clause gadget with no outer edge covered, so we need to pick all three inner vertices, pushing us to $\geqslant n + 2m + 1$.

Theorem 7.3 (Ladner)**.** *If* $\mathsf{P} \neq \mathsf{NP}$*, then there is a problem in* NP *that is neither in* P *nor* NP*-complete.*

Proof. Let $f : \mathbb{N} \to \mathbb{N}$ be a function to be defined. Then let $g : \{0,1\}^* \to \{0,1\}$ be defined as:

If $u \in \{0,1\}^*$ has the form $x01^{f(|x|)}$, and x encodes a satisfiable formula (in some sensible way), then $g(u) = 1$, otherwise $g(u) = 0$.

We shall define functions $i : \mathbb{N} \to \mathbb{N}$ and $f : \mathbb{N} \to \mathbb{N}$ recursively as follows: Enumerate all Turing machines as $T_1, T_2, \ldots$ in such a way that each Turing machine occurs infinitely often. Also, constrain T_j to halt within $j \cdot n^j$ steps.

Observe that if T is any Turing machine that halts in polynomial time, then it agrees with some T_j. If we have defined $i(1), \ldots, i(n-1)$, $f(1), \ldots, f(n-1)$, then we set $i(n)$ to be $i(n-1)$ if $T_{i(n-1)}$ agrees with g on all inputs of size $\leqslant \log n$, and as long as $i(n-1) < \log\log n$, set it to $i(n-1)+1$ otherwise, then

$f(n) = i(n) \cdot n^{i(n)}$.

Note that this isn't a circular definition, as we have enough values of f to compute g on inputs of size $\leqslant \log n$. Note also that f is computable in polynomial-time (in n), since we can do a brute-force search.

Claim 1: $g \notin \mathsf{P}$.
Proof. If $g \in \mathsf{P}$, then g is computed by some T_j, but then $i(n)$ never goes above j, so $f(n) \leqslant jn^j$ for all n. But then the amount of padding is polynomial in $|x|$, so if we can compute g in polynomial time, then SAT$\in \mathsf{P}$, which contradicts the assumption that $\mathsf{P} \neq \mathsf{NP}$.

Claim 2: g is not NP-complete. *Proof.* If g is NP-complete, then we have a polynomial time reduction of SAT to g. If the reduction is in time kn^k, then the reduction is to an instance u of g of size at most kn^k. But for n sufficiently large $i(n)n^{i(n)} > kn^k$ which implies that the "x part" of u has size less than n. So in polynomial time, we reduce an instance of SAT to a smaller instance of SAT, so SAT$\in \mathsf{P}$, but $\mathsf{P} \neq \mathsf{NP}$. □

7.1 Completeness in space classes

Definition 7.4 (DSPACE/NSPACE)**.** Let $s : \mathbb{N} \to \mathbb{N}$, then $\mathsf{DSPACE}(s(n))$ is the class of functions computable by a Turing machine in space $O(s(n))$. $\mathsf{NSPACE}(s(n))$ is the same thing, but with a NDTM.

Theorem 7.5 (Savitch)**.** *If $s(n)$ grows at least as fast as $\log n$ ($s(n) \geqslant c \log n$ for $c > 0$) then*

$$\mathsf{NSPACE}(s(n)) \subset \mathsf{DSPACE}(s(n)^2)$$

Proof. Let $f \in \mathsf{NSPACE}(s(n))$, let T be a NDTM that computes f in space $s(n)$, and for any x let G_x be the configuration graph of T with input x. Note that $|V(G_x)| = 2^{O(s(|x|))}$ (since $s(n) \geqslant c \log n$). Then for any x, $f(x) = 1$ iff there is a path in G_x from the initial configuration, v_{init} to the "halt-with-output-1" vertex v_{accept} (we add a vertex v_{accept} which is joined to all configurations where T has halted with output 1).

Lecture 9 Let $\sigma(k)$ be the amount of space needed to determine whether there is a path of length $\leqslant 2^k$ between two vertices of the configuration graph. and let $g_k(u, v) = 1$ if such a path exists from u to v. Then $g_k(u, v) = 1$ if and only if there exists w such that $g_{k-1}(u, w) = 1$ and $g_{k-1}(w, v) = 1$ (i.e. there is a path of length 2^k if and only if there are two intermediate paths of length 2^{k-1}). Therefore $\sigma(k) \leqslant O(s(n)) + \sigma(k-1) + 1$ (by reusing space). Also, $\sigma(0) = O(s(n))$, so

$\sigma(k) = O(k \cdot s(n))$. Choose $k = O(s(n))$ such that $2^k > |V(G_x)|$ and we get the bound we wanted. □

7.2 A PSPACE-complete problem

Definition 7.6 (Quantified Boolean Formula)**.** A **quantified boolean formula** is a statement of the form

$$Q_1 x_1 Q_2 x_2 \dots Q_n x_n \phi(x_1, \dots, x_n),$$

where ϕ is built out of $\neg$, $\wedge$, $\vee$ and each Q_i is $\exists$ or $\forall$.

TQBF is the problem of determining whether a QBF is true. It's basically like playing a game, as long as the quantifiers alternate.

Theorem 7.7. *TQBF is* PSPACE*-complete.*

Proof. First, let's show that TQBF is actually in PSPACE. Let $S(n, m)$ be the amount of space needed to solve TQBF for formulae of size m with n variables, where constants are allowed. Let P be the QBF $Q_1 x_1 \dots Q_n x_n \phi(x_1, \dots, x_n)$ where ϕ has size m.

If $Q_1 = \forall$, then we get that P is true iff $P_{x_1=1}$ is true, and $P_{x_1=0}$ is true. The space needed to determine whether this is the case is at most $S(n-1, m) + 1 + O(m)$ (by reusing space). Also $S(0, m) = O(m)$ since you just evaluate a formula where every variable is known. It is similar if $Q_1 = \exists$. Therefore $S(n, m) = O(n \cdot m)$. Therefore TQBF is in PSPACE.

Now let f be any other function in PSPACE. We want a polynomial time reduction of f to TQBF. Let $x \in \{0, 1\}^*$ be some string, and let T be a Turing machine that computes f in polynomial space, and let G be the configuration graph of T with input x. Then $f(x) = 1$ iff there is a path in G from v_{init} to v_{accept}. Observe that $|V(G)| = 2^{O(p(n))}$ for some polynomial p.

Now, for any k, u, v, let $Q(k, u, v)$ be the statement that there is a path in G of length $\leqslant 2^k$ from u to v. Let $S(k)$ be the maximum size of a QBF that expresses $Q(k, u, v)$.

Then, $Q(k, u, v) \Leftrightarrow \exists w Q(k-1, u, w) \wedge Q(k-1, w, v)$. Then, this is true if and only if $\exists w \forall y, z\ ((y = u \wedge z = w) \vee (y = w \wedge z = v)) \Rightarrow Q(k-1, y, z)$. We write it down like this so that we only contain one formula of size $S(k-1)$. Then it follows that $S(k) \leqslant S(k-1) \leqslant O(p(n)) + S(k-1)$ (because the terms in the formula preceding $Q(k-1, y, z)$ that look like $y = u$ are actually shorthand

for $y_1 = u_1 \wedge y_2 = u_2 \dots$). Therefore we get $S(k) = O(p(n)^2) + S(0)$. But $Q(0, u, v) = 1$ iff $u = v$ or $uv \in E(G)$. We can find, in polynomial time, a formula of size polynomial in n that determines whether one configuration is the successor of another - see proof that $\mathsf{P} \subset \mathsf{P/poly}$ where we do this. □

7.3 NL-completeness

Definition 7.8 (Implicitly logspace computable)**.** A boolean function $f : \{0,1\}^* \to \{0,1\}^*$ is **implicitly logspace computable** (or we can write ILC) if for every x, and every i, the functions $\mathbb{1}_{\{i \leqslant |f(x)|\}}$ and $f(x)_i$ - the ith bit of $f(x)$ - are in L (the latter only if $i \leqslant |f(x)|$). We can compose the Turing machines that do this, and feed the Turing machine a small sequence telling it what we want it to do, in order to get a single Turing machine which computes all of this stuff.

Lemma 7.9. *If f and g are implicitly logspace computable, then so is $g \circ f$.*

Lecture 10

Proof. Start computing $g(f(x))$, "pretending" that we have an input tape with $f(x)$ written on it. T_g is the Turing machine that computes g in logspace, and T_f is one that computes f in logspace. Then each time T_g needs to read a bit of $f(x)$, we use T_f to compute that bit. (Also, if T_g needs to know where $f(x)$ ends, we use the fact that $\mathbb{1}_{\{i \leqslant |f(x)|\}}$ is also in L.) In this way, we compute any given bit of $g(f(x))$ in space essentially equal to the sum of space needed by T_g and space needed by T_f, and in particular space $O(\log n)$. A similar trick works for computing $\mathbb{1}_{\{i \leqslant g(f(x))\}}$. □

Definition 7.10 (Logspace reducible)**.** Let $f, g : \{0,1\}^* \to \{0,1\}$. Then g is **logspace reducible** to f if there is a function $h : \{0,1\}^* \to \{0,1\}^*$ which is ILC, such that $g = f \circ h$.

If g is logspace reducible to f, we write $g \prec_{\mathsf{L}} f$.

Definition 7.11 (NL-hard, NL-complete)**.** We say that f is **NL-hard** if every $g \in \mathsf{NL}$ is logspace reducible to f. We say that f is **NL-complete** if $f \in \mathsf{NL}$, and f is NL-hard.

Lemma 7.12. *1. If $h \prec_{\mathsf{L}} g$ and $g \prec_{\mathsf{L}} f$, then $h \prec_{\mathsf{L}} f$.*

2. If f is NL-complete, $g \in \mathsf{NL}$, and $f \prec_{\mathsf{L}} g$, then g is NL-complete.

Proof. The proofs are obvious from the lemma about composition of implicitly logspace computable functions. □

Example. An NL-complete function:

Proposition 7.13. *STCONN is NL-complete.*

Proof. Let $f \in \mathsf{NL}$, let T be an NDTM that computes f, let $x \in \{0,1\}^*$, and let G be the configuration graph of T with input x. Then G is seen to be implicitly logspace computable, since to check whether two vertices are connected, I just need to follow through the path, and each time I move one vertex, I can forget where I am, and just remember the new vertices. We can also check if we reach the end of the computation at each step.

But $f(x) = 1$ iff there is a directed path in G from v_{init} and v_{accept}. So we have a logspace reduction of f to STCONN.

7.4 P-completeness

Obviously, we aren't going to use polynomial time reductions, because then P-completeness is completely trivial.

Definition 7.14 (P-completeness)**.** A function f in P is **P-complete** if ever $g \in \mathsf{P}$ is logspace reducible to f.

Lemma 7.15. $\mathsf{NL} \subset u - \mathsf{AC}^1$.

Proof. Let $f \in \mathsf{NL}$, $x \in \{0,1\}^*$ and T a NDTM that computes f in logspace. Then examining the proof that $\mathsf{P} \subset \mathsf{P/poly}$, we see that there is a circuit of depth $O(1)$ which inputs a pair of configurations (κ, κ'), and outputs 1 iff (κ, κ') belongs to the configuration graph. Moreover, this circuit is implicitly logspace computable.

It follows that the (directed) adjacency matrix A of G is implicitly logspace computable. Also, the number of configurations is $2^{O(\log n)}$, so polynomially bounded. Let $p(n)$ be a polynomial upper bound. Then $f(x) = 1$ iff there is a path of length $\leqslant p(n)$ from v_{init} to v_{accept}.

Given 2 matrices with values in $\{0,1\}$, we define their **boolean product** AB by

$$(AB)_{ij} = \bigvee_k A_{iK} \wedge B_{kj}.$$

(We need A to have the same set of columns as B has rows). If A and B are square matrices with 1s along the diagonal, then $AB_{ij} = 1$ iff either $i = j$ or there is a path from i to j (possibly with loops) with first edge in A and second in B. (Here A and B are adjacency matrices of directed graphs). Therefore, for a square matrix, $(A^r)_{ij} = 1$ iff there is a path of length $\leqslant r$ from i to j. So it follows that we are done if we can find a depth $O(\log n)$ circuit for computing $A^p(n)$ (where A is the adjacency matrix of G.)

From the definition of Boolean product, there is a circuit of depth 2 for computing it (it's also incredibly simple, and easily seen to be implicitly logspace computable). By repeated squaring , we get a circuit of depth $O(\log n)$ for computing $A^{p(n)}$. It also has polynomial size. □

Corollary 7.16. *If f is P-complete, then if $f \in u - \mathsf{NC}$, then $\mathsf{P} = \mathsf{NC}$.*

Proof. Suppose $f \in u - \mathsf{NC}$. Let $g \in \mathsf{P}$. Then g is logspace reducible to f, so there exists a function h which is ILC such that $g = f \circ h$. Since $\mathsf{L} \subset u - \mathsf{AC}^1 \subset u - \mathsf{NC}^2$, h can be computed by a logspace uniform family of circuits of depth $O((\log n)^2)$. Then if we use this family of circuits to compute h, we compose these circuits with the circuits that compute f, we get $g \in \mathsf{NC}$. (We can actually do slightly better, by picking a specific $u - \mathsf{NC}^i$, since we only add an $O((\log n)^2)$ circuit). □

Definition 7.17 (CIRCUITEVAL)**. CIRCUITEVAL** is the problem which takes as input a circuit C with n inputs, and $x \in \{0,1\}^n$, and outputs $C(x)$.

Proposition 7.18. *CIRCUITEVAL is P-complete.*

Proof. Obviously CIRCUITEVAL is in P. Also, if $f \in \mathsf{P}$ and f is computable by T, then there is an ILC family (C_n) of circuits that emulate T (where C_n has n inputs). So

$$f(x) = 1 \Leftrightarrow \text{CIRCUITEVAL}(C_{|x|}, x) = 1.$$

So f is logspace reducible to CIRCUITEVAL. □

8 Less obvious relationships between complexity classes

Definition 8.1 (Nice function)**.** We say that $f : \mathbb{N} \to \mathbb{N}$ is **nice** if it satisfies the following:

1. $f(n) \geqslant n$ for every n.
2. $f(n)$ is increasing.
3. The function $1^n \mapsto f(n)$ can be computed in time $O(f(n))$.

The next theorem is a weakening of a result called the **Time Hierarchy Theorem**.

Theorem 8.2. *There exists a polynomial p such that for every nice function f, there is a function ϕ that can be computed in time $p(f(n))$, such that if ψ is any function that can be computed in time $f(n)$, then there are infinitely many x such that $\phi(x) \neq \psi(x)$.*

Proof. The basic iea is as follows: We define $\phi(x)$ to be $1 - T(x)$ if x encodes the Turing machine T and T halts in time $f(|x|)$. Otherwise, let $\phi(x) = 0$.

Organise the encoding in such a way that if $s = t0^m$, and one of s, t encodes T, then so does the other. Now suppose that T is some Turing machine that halts in time $f(n)$. If t encodes T, then $\phi(t) = 1 - T(t)$, so ϕ disagrees with T on t, but by the way we estalished our encoding, t can be arbitrarily long, so ϕ and T disagree infinitely often. So ϕ cannot be computed by any Turing machine that holds in time $f(n)$.

It remains to prove that ϕ can be computed in time $p(f(n))$ for some polynomial p. To comoute ϕ, we simulate T using a universal Turing machine, and the encoding t. It is not hard to see that if T takes time m, then the simulation takes time $O(m^2)$ (for each step of the Turing machine T, the universal Turing machine will need to do that step, then walk all the way back to the beginning of the work tape where the TM is stored, and the furthest it could ever need to walk is m spaces, and it needs to do this at most m times, then computing the turing machines output takes time $O(m)$, and "figuring out what step T performs next" takes a constant overhead). So we run the universal turing machine for time $O(f(n)^2)$ until the simulated machine has taken $f(n)$ steps. That is enough to compute $\phi(x)$ when $|x| = n$. □

Can we use diagonalisation to prove $\mathsf{P} \neq \mathsf{NP}$. Well, it turns out, it wouldn't be easy.

Definition 8.3 (Oracle Turing Machine)**.** Let $A : \{0,1\}^* \to \{0,1\}$. An **Oracle Turing machine with oracle A** is a TM with an extra **oracle tape**, and at any time it is allowed to "query the oracle." That is, if y is the content of the oracle tape, it can obtain the value of $A(y)$ in *one step*!

Given any uniform complexity class, the "A version" is what you get when you get by replacing a Turing machine with an Oracle Turing machine with oracle A.

Examples.

- P^A are functions computable by an OTM (Oracle Turing machine) with oracle A in polynomial time.
- NP^A are functions computable by a NDOTM with oracle A in polynomial time. (Or, equivalently the certificate version still works).

We shall now find oracles A, B such that $\mathsf{P}^A = \mathsf{NP}^A$, but $\mathsf{P}^B \neq \mathsf{NP}^B$.

Let $\mathrm{EXPCOM}(T, x, 1^n) = T(x)$ if T computes $T(x)$ in time 2^n.

Proposition 8.4. $\mathsf{P}^{EXPCOM} = \mathsf{NP}^{EXPCOM} (= \mathsf{EXPTIME})$.

Proof. Let $f \in \mathsf{EXPTIME}$ and suppose that $f(x)$ can be computed in time $2^{p(|x|)}$ for each x by a Turing machine T. Then given x, create the triple $(T, x, 1^{p(|x|)})$ on the oracle tape, and query the oracle. This shows that $f \in \mathsf{P}^{\mathrm{EXPCOM}}$. Clearly $\mathsf{P}^{\mathrm{EXPCOM}} \subset \mathsf{NP}^{\mathrm{EXPCOM}}$. Now let $f \in \mathsf{NP}^{\mathrm{EXPCOM}}$, and let $g \in \mathsf{P}^{\mathrm{EXPCOM}}$ and q be such that

$$p(x) = 1 \Leftrightarrow \exists y.\, |y| = q(|x|), \text{ and } g(x, y) = 1.$$

Each time g queries EXPCOM, the oracle tape contains at most polynomial many bits, so we can carry out the computation in time $2^{\mathrm{poly}(n)}$. At most polynomially many queries are made of A, so $g \in \mathsf{EXPTIME}$. But we can calculate $g(x, y)$ for every y by running $2^{q(n)}$ of these computations. So $f \in \mathsf{EXPTIME}$. □

Proposition 8.5. *There exists a B such that* $\mathsf{P}^B \neq \mathsf{NP}^B$.

Proof. Suppose that we've chosen B. Then define a function ϕ by

$$\phi(x) = 1 \Leftrightarrow \exists y. \text{ s.t. } |y| = |x| \text{ and } B(y) = 1.$$

Then $\phi \in \mathsf{NP}^B$.

Lecture 11

Lecture 12

Lecture 13

9 Two randomized algorithms

9.1 Polynomial identity testing

The input of this problem are polynomials P and Q over a field $\mathbb{F}$, and the output is 1 if P and Q are the same polynomial. Typically we present these polynomials as arithmetic circuits for calculating polynomials (which correspond to a presentation of the polynomial), and determining whether these circuits compute the same polynomial.

One way to prove two polynomials are *not* the same is two compute their result on some values, and if they differ then the polynomials must be differ. Of course, we could get unlucky, but how likely are we to get unlucky?

Lemma 9.1 (Schwartz-Zippel, also DeMillo-Lipton)**.** *Let* P *be a polynomial of total degree* d *in* n *variables* $x_1, \ldots, x_n$ *over a field* $\mathbb{F}$*, and let* $S \subset \mathbb{F}$*. Then the number of* $s \in S^n$ *such that* $P(s) = 0$ *is at most* $d|S|^{n-1}$*. This bound is sharp unless* $d > |S|$*.*

□

Now the algorithm for determining if two polynomials are the same is to consider a polynomial of degree $d < |\mathbb{F}|$[3], choose $S \subset \mathbb{F}$ such that $d < |S|$ (if possible, otherwise the algorithm doesn't work).

Then randomly pick $s \in S^n$, and compute $P(s)$ and $Q(s)$[4], if $P(s) = Q(s)$, then output $P = Q$, otherwise output $P \neq Q$. If $P = Q$ then the algorithm outputs $P = Q$ with probability 1, and if $P \neq Q$ it outputs $P \neq Q$ with probability at least $1 - \frac{d}{|S|}$ (by the Schwartz-Zippel lemma applied to $P - Q$).

If $1 - \frac{d}{|S|} < \frac{1}{2}$, then repeat k times to boost to $1 - (\frac{d}{|S|})^k$. choosing k appropriately. This shows that polynomial identity testing is in coRP.

[3] which we can choose by $x^p = x$ i.e. Fermat's little theorem

[4] we assume P, Q are presented in a way that makes this computation efficient, i.e. not as something like the permanent of a matrix

9.2 Primality Testing

The problem takes input n and outputs 1 if and only if n is prime. We shall show that this is in coRP.

Lemma 9.2. *The polynomials $(X+1)^n$ and X^n+1 are equal (as polynomials) mod n iff n is prime.*

Proof. If n is prime, then

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}$$

is divisble by n, since $n \mid n!$, and $n \nmid k!$, $n \nmid (n-k)!$. If n is composite, pick a prime p such that $p \mid n$. Then

$$\frac{1}{n}\binom{n}{p} = \frac{(n-1)\cdot(n-2)\cdot\cdots\cdot(n-p+1)}{p!},$$

which is not an integer, since p divides the denominator but not the numerator. □

We can't get anywhere by testing for identity of $(X+1)^n$ and X^n+1, e.g. if n is a Carmichael number[5] and n is not prime, then they will take the same values everywhere (but be different as polynomials). Instead we do this:

- Pick a random monic polynomial Q of degree d ($\sim 2\log n$), with coefficients in $\mathbb{Z}/n\mathbb{Z}$.
- Calculate $(X+1)^n$ and X^n+1 in the ring $\frac{\mathbb{Z}}{n\mathbb{Z}}[X]/Q(X)$.
- If they are equal, output "n is prime", else output "n is composite."

The second step asks if $(X+1)^n$ and X^n+1 are congruent $\bmod n$. and $\bmod Q(X)$. We can test this by repeated squaring in time $O(\log n)\cdot O(d^2)$ which is polynomial in $\log n$.

If n is prime, then $(X+1)^n$ and X^n+1 are the same *polynomial* $\bmod n$, so they must be the same $\bmod Q(X)$ for all Q. So the algorithm will output that n is prime.

To deal with the case where n is composite, we will need to quote a number theory lemma

Lemma 9.3. *Let p be a prime, and let d be a positive integer. Then over $\mathbb{F}_p$, the polynomial $X^{p^d}-X$ is the product of all irreducible monic polynomials of degree dividing d.*

[5] i.e. $\forall a.\, a^n \equiv a$

□

Corollary 9.4 (T)**.** *he number of monic irreducible polynomials of degree d over $\mathbb{F}_p$ is $(1 - o(1))\frac{p^d}{d}$. (So as a fraction, approximately $\frac{1}{d}$ polynomials are irreducible).*

Proof. Let $\phi(k)$ be the number of monic irreducible polynomials of degree k. Then by the lemma,

$$p^d = \sum_{k|d} k\phi(k).$$

So by Möbius inversion,

$$d\phi(d) = \sum_{k|d} \mu(k) p^{d/k},$$

where μ is the Möbius function, defined by

$$\mu(p_1 \cdot \cdots \cdot p_k) = \begin{cases} (-1)^k, & \text{if } p_1, \ldots, p_k \text{ all distinct primes} \\ 0, & \text{otherwise.} \end{cases}$$

The leading term is p^d, and remaining terms at most $p^{d/2}$, so result follows. Suppose now that n is composite, and let p be a prime that divides n. If $\mathbb{P}$ [algorithm outputs "n is composite."] $\leqslant \frac{1}{2d}$, then the probability that $Q(X)$ is irreducible, and $Q(x) \mid (X+1)^n - X^n - 1$ in $\mathbb{F}_p[X]$ is at least $\frac{1}{2d} - o(1)$. But any two distinct monic irreducible polynomials are coprime, so this shows that

$$d\left(\frac{1}{2d} - o(1)\right) p^d \leqslant n.$$

This is a contradiction if $d \geqslant 2\log n$. So if n is composite and $d \geqslant 2\log n$, then the

$$\mathbb{P}(\text{alg. outputs "}n\text{ is composite"}) \geqslant \frac{1}{2d} - o(1).$$

By repeating $O(d)$ times, we can boost this to $1/2$.